



School of Future Tech

Case Study Report

on

API Rate Limiter Simulation

by

Satvik Ashok Poojary

150096725106

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Modern web applications frequently interact with APIs to fetch or send data. However, uncontrolled and excessive API calls can overload servers, reduce performance, and cause system instability. In high-traffic systems, it is essential to regulate how often a function or API can be executed.

This case study focuses on designing and implementing an **API Rate Limiter Simulation using JavaScript**. The system demonstrates how throttling logic can restrict the execution of a function to a specified number of calls per second. The project simulates real-world API rate limiting behavior in a browser environment and visually shows allowed and blocked function calls.

The objective is to understand timing control, throttling mechanisms, closures, and dynamic DOM manipulation in modern JavaScript applications.

2. Problem Statement / Case Background (Abstract)

Background

In real-world applications, users may trigger actions repeatedly (e.g., clicking a button rapidly or refreshing a page multiple times). If every action results in an API call, the server may experience:

- Excessive load
- Resource exhaustion
- Slower response times
- Potential system failure

Therefore, systems implement **rate limiting** to control how many requests can be processed within a specific time frame.

Abstract

This case study presents the design and implementation of a browser-based **API Rate Limiter Simulation** using JavaScript. A reusable `throttle(fn, limit)` function is developed to restrict function execution to a defined number of calls per second. The system uses timestamp tracking (`Date.now()`), execution windows, and closure-based state management to enforce rate limits.

The application provides a user interface where users can configure the rate limit, simulate rapid API calls, and observe allowed versus blocked executions in real time. The system demonstrates how throttling ensures controlled execution frequency and prevents excessive function calls.

3. Case Study Design

The system is designed as a simple web-based simulation consisting of:

1. User Interface Components

- Input field to set rate limit (calls per second)
- Button to simulate API call
- Display for allowed call count
- Display for blocked call count
- Log section showing timestamps

2. Core Functional Components

- Reusable throttle function
- Time window management (1 second interval)
- Call counting mechanism
- Dynamic DOM updates
- Event listeners for user interaction

3. Execution Flow

1. The user enters a rate limit (e.g., 3 calls/sec).
2. The user clicks the "Simulate API Call" button repeatedly.
3. The throttle function:
 - Allows calls within the limit.
 - Blocks calls exceeding the limit.
4. After 1 second, the counter resets.
5. Results are displayed dynamically without page reload.

This design demonstrates practical implementation of rate limiting logic in a real-time interactive environment.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

1. Throttling Algorithm

A custom throttling algorithm was implemented using:

- A counter to track number of calls
- Timestamp comparison to manage time window
- Conditional checks to allow or block execution
- Automatic reset after interval expiration

2. Timestamp-Based Execution Window

The system uses:

`Date.now()`

to:

- Capture the starting time of the execution window
- Compare current time with window start
- Reset the counter after 1000 milliseconds

3. Call Counting Mechanism

A variable `callCount` tracks the number of allowed calls within the current time window. If the count reaches the specified limit, further calls are blocked until the window resets.

4. Closure (Temporal State Management)

The throttle function uses closures to preserve:

- `callCount`

- `windowStart`

This ensures that state persists across multiple button clicks.

5. Higher-Order Function

The throttle function:

- Accepts another function as input
- Returns a new wrapped function

This demonstrates higher-order function design in JavaScript.

6. ES6 Concepts Used

- Arrow functions
- Template literals
- Closures
- Higher-order functions
- Spread operator (`...args`)

7. Technology Stack

- HTML – Structure
- CSS – Styling and layout
- JavaScript (ES6) – Logic and execution control
- Browser environment – Execution platform

5. Problem Statement / Case Study Implementation Details and Snapshots.

Implementation Files

1. `index.html`

- Contains UI structure.
- Input field, button, counters, and log section.

2. `style.css`

- Provides clean layout and styling.
- Ensures readable and structured interface.

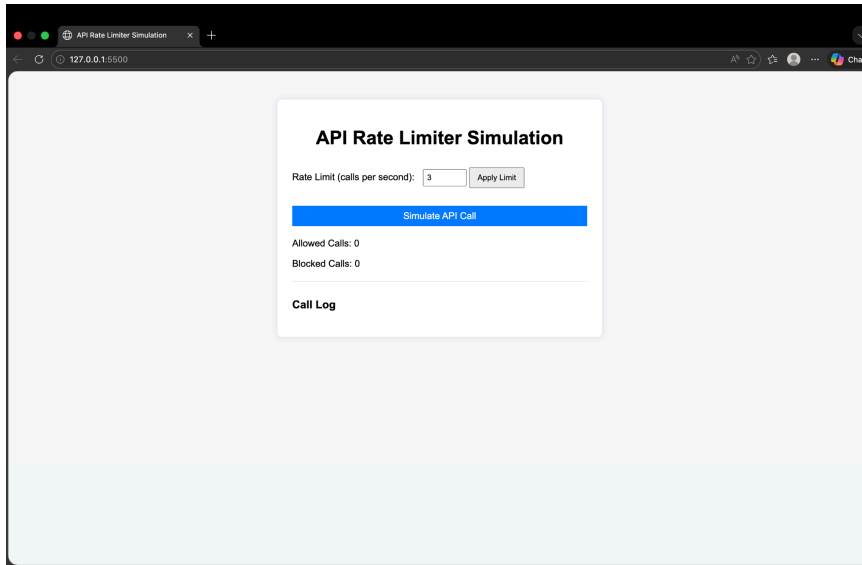
3. `script.js`

- Contains throttle logic.
- Implements call counting and reset mechanism.
- Handles dynamic DOM updates.

Key Implementation Logic

- When the button is clicked:
 - The throttled function checks if the call count is below the limit.
 - If yes → execute and log as allowed.
 - If no → block and log as blocked.
- After 1 second:
 - Reset counter.
 - Start a new execution window.

Screenshot of the User Interface



Screenshot Showing Rapid Clicks with Blocked Calls And Screenshot of Allowed vs Blocked Counters

API Rate Limiter Simulation

Rate Limit (calls per second):

Allowed Calls: 7

Blocked Calls: 6

Call Log

- ✓ Allowed at 11:47:57 AM
- ✗ Blocked at 11:47:47 AM
- ✓ Allowed at 11:47:47 AM
- ✓ Allowed at 11:47:47 AM
- ✓ Allowed at 11:47:47 AM
- ✗ Blocked at 11:47:46 AM

Screenshot After Changing Rate Limit

API Rate Limiter Simulation

Rate Limit (calls per second): Apply Limit

Simulate API Call

Allowed Calls: 2

Blocked Calls: 2

Call Log

- ✗ Blocked at 11:52:09 AM
- ✗ Blocked at 11:52:09 AM
- ✓ Allowed at 11:52:09 AM
- ✓ Allowed at 11:52:08 AM
- 🔄 Rate limit updated to 2 calls/sec

6. Problem Statement / Case Study Results and Conclusion.

Results

The system successfully demonstrates:

- Real-time control of function execution.
- Accurate timestamp-based execution windows.
- Proper reset logic after a time interval.
- Clear differentiation between allowed and blocked calls.
- Dynamic adjustment of rate limits.

The simulation effectively replicates real-world API rate limiting behavior in a controlled environment.

Conclusion

This case study demonstrates a complete implementation of throttling logic using JavaScript. The system successfully integrates:

- Execution timing control
- Closure-based state management
- Higher-order function design
- Real-time user interaction

The project highlights the importance of rate limiting in modern web systems to prevent excessive requests and ensure performance stability. The implementation provides a strong foundation for understanding advanced concepts such as debouncing, distributed rate limiting, and backend API protection mechanisms.

7. References

- Official documentation of JavaScript from Mozilla Developer Network (MDN) for understanding closures, higher-order functions, and execution context.
- MDN Web Docs – Documentation for `Date.now()` and JavaScript timing methods used for timestamp tracking and execution windows.
- JavaScript ES6 documentation for arrow functions, template literals, and modern function syntax.
- Web development resources and tutorials on REST API rate limiting and throttling techniques.
- Articles and technical blogs on API rate limiting concepts and real-world implementation strategies in frontend applications